

Real-world (rebuild a Project 1 feature, LCEL-style) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `lcel_vs_raw_sdk_practice.py` : reading about "LangChain vs. raw SDK" trade-offs isn't the same as seeing both versions of the exact same feature side by side. Rebuilding one real Project 1 feature with LCEL, then diffing the outputs, is what makes the comparison real instead of theoretical. **If not:** the Build Task's `compare_with_raw_sdk.py` would be the first time you ever built this comparison, with no smaller version to trust it against.

All examples below assume `from project_1 import extract_raw, ExtractedData` names your real Project 1 structured-extraction function and Pydantic model.

Basic Version

Approach 1 — print and eyeball

```
# lcel_vs_raw_sdk_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

prompt = ChatPromptTemplate.from_template(
    "Extract structured data from: {text}"
)
lcel_chain = prompt | ChatOpenAI().with_structured_output(ExtractedData)

test_inputs = ["input one text", "input two text", "input three text"]

for text in test_inputs:
    raw_result = extract_raw(text)
    lcel_result = lcel_chain.invoke({"text": text})
    print("Input:", text)
    print("  raw: ", raw_result)
    print("  lcel:", lcel_result)
```

This shows both results side by side. It leaves the comparing to your own eyes, which doesn't scale past 3 inputs and is easy to get wrong — see Intermediate.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — actually assert the match

```
# lcel_vs_raw_sdk_practice.py
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

def build_lcel_chain(model_name: str = "gpt-4o-mini"):
    prompt = ChatPromptTemplate.from_template(
        "Extract structured data from: {text}"
    )
    model = ChatOpenAI(model=model_name, temperature=0)
    return prompt | model.with_structured_output(ExtractedData)

def compare(text: str, lcel_chain) -> bool:
    # why: == on two Pydantic objects checks every field – the only real
    # way to prove the raw and LCEL versions agree, not just look similar.
    raw_result = extract_raw(text)
    lcel_result = lcel_chain.invoke({"text": text})
    match = raw_result == lcel_result
    print(f"{text!r} -> match: {match}")
    if not match:
        print(f"  raw: {raw_result}")
        print(f"  lcel: {lcel_result}")
    return match

def main() -> None:
    lcel_chain = build_lcel_chain()
    test_inputs = ["input one text", "input two text", "input three text"]
    results = []
    for text in test_inputs:
        results.append(compare(text, lcel_chain))
    print(f"\n{sum(results)}/{len(results)} matched")
```

```
if __name__ == "__main__":
    main()
```

Expected output:

```
'input one text' -> match: True
'input two text' -> match: True
'input three text' -> match: True

3/3 matched
```

Difference from Basic: computing `match = raw_result == lcel_result` and reporting a final `N/M matched` count actually proves the comparison, instead of leaving it to a human reading printed text. Full type hints and a `build_lcel_chain()` function also make this reusable.

Approach 2 — results collected into a typed list, `run_comparison()` split from `main()`

```
# lcel_vs_raw_sdk_practice.py
from dataclasses import dataclass
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from project_1 import extract_raw, ExtractedData

@dataclass
class ComparisonRow:
    # why: one typed record per input, instead of printing inline – this
    # is what makes run_comparison()'s result reusable by a caller or a test.
    input_text: str
    raw_result: ExtractedData
    lcel_result: ExtractedData
    matched: bool

def build_lcel_chain(model_name: str = "gpt-4o-mini"):
    prompt = ChatPromptTemplate.from_template(
        "Extract structured data from: {text}"
    )
    model = ChatOpenAI(model=model_name, temperature=0)
    return prompt | model.with_structured_output(ExtractedData)

def run_comparison(test_inputs: list[str]) -> list[ComparisonRow]:
    # why: the logic (run both versions, record whether they matched) lives
    # here, separate from main()'s reporting – this is the shape the Build
```

```

# Task's compare_with_raw_sdk.py copies almost unchanged.
lcel_chain = build_lcel_chain()
rows = []
for text in test_inputs:
    raw_result = extract_raw(text)
    lcel_result = lcel_chain.invoke({"text": text})
    matched = raw_result == lcel_result
    rows.append(ComparisonRow(text, raw_result, lcel_result, matched))
return rows

def main() -> None:
    test_inputs = ["input one text", "input two text", "input three text"]
    rows = run_comparison(test_inputs)

    matched = 0
    for row in rows:
        if row.matched:
            matched = matched + 1
    print(f"{matched}/{len(rows)} matched")
    for row in rows:
        if not row.matched:
            print(f"MISMATCH on: {row.input_text}")
            print(f"  raw: {row.raw_result}")
            print(f"  lcel: {row.lcel_result}")

if __name__ == "__main__":
    main()

```

Expected output:

```
3/3 matched
```

Difference from Approach 1: Approach 1 proves the two versions agree and stops there — the comparison logic and the printing are tangled together in one function. Approach 2 splits `run_comparison()` (the logic, returns data) from `main()` (the reporting), and collects results into a typed `ComparisonRow` list instead of printing inline — this is what makes the script reusable elsewhere with no changes, which is exactly the shape this document's Build Task needs.

Which one should you actually run? Approach 1 is enough for a quick, one-time check. Approach 2's structured version is worth it the moment you expect to re-run this comparison regularly (every

time you touch the prompt, or upgrade a LangChain version) — the `run_comparison() / main()` split means the exact same logic could later be called from a real test, not just read by a human.